

Simulazioni d'Esame: Fondamenti di Programmazione 2

Simulazione 1

Esercizio 1

Data la seguente porzione di programma rispondere, dando motivazione, alle domande corrispondenti:

```
void f(int*& p) {
    p++;
}

int main() {
    int* matricola = new int[6] { /* Inserisci tua matricola */ };

    // 1. Le istruzioni seguenti sono corrette? Se si, cosa stampano?
    int* q = matricola;
    f(q);
    cout << *q << endl;

    // 2. Le istruzioni seguenti sono corrette? Se si, cosa stampano?
    int& r = matricola[3];
    r += 2;
    cout << matricola[3] << endl;

    // 3. La seguente istruzione e' corretta? Motivare la risposta.
    *(matricola + matricola[0]) = 0;

    // 4. Scrivere sul foglio le istruzioni per deallocare la memoria
    // dinamica allocata nel programma.
    return 0;
}
```

Esercizio 2

Modellare e implementare una classe **FlottaAziendale** che gestisca i veicoli di un'azienda. Ogni veicolo è caratterizzato da una targa (stringa), un modello (stringa) e un chilometraggio (intero). Si richiede di implementare le seguenti funzionalità:

- Aggiungere un nuovo veicolo alla flotta. Se un veicolo con la stessa targa è già presente, l'inserimento deve essere ignorato e il metodo deve restituire **false**.
- Un metodo per aggiornare il chilometraggio di un veicolo (ricercato per targa). Il chilometraggio può solo aumentare o restare invariato; se viene fornito un valore inferiore, l'aggiornamento deve essere rifiutato.
- Il modello e la targa di un veicolo, una volta inseriti, devono essere immutabili.
- Definire un operatore di confronto tra due veicoli, ritenuti uguali se e solo se hanno la stessa targa.
- Un metodo **veicoli_da_revisionare** che, preso in input un intero k , restituisca un **vector<string>** contenente le targhe di tutti i veicoli che hanno superato i k chilometri.

Simulazione 2

Esercizio 1

Data la seguente porzione di programma rispondere alle domande corrispondenti:

```
int main() {
    int* matricola = new int[6] { /* Inserisci tua matricola */ };

    // 1. La seguente istruzione e' corretta? Se si, cosa stampa?
    int a = matricola[1];
    int& b = matricola[1];
    a++;
    b++;
    cout << matricola[1] << " " << a << endl;

    // 2. Le istruzioni seguenti sono corrette? Se si, cosa stampano?
    int* p = new int[3];
    for (int i = 0; i < 3; i++) {
        p[i] = matricola[5 - i];
    }
    cout << *(p + 2) << endl;

    // 3. Perche' la seguente sequenza genera un memory leak?
    int* temp = new int[6];
    temp = matricola;

    // 4. Scrivere TUTTE le istruzioni necessarie per deallocare la
    // memoria allocata fino a questo punto (correggendo il punto 3).
    return 0;
}
```

Esercizio 2

Si vuole implementare un sistema per la gestione dei prestiti in una biblioteca. Implementare una classe `GestioneBiblioteca` che memorizzi i libri disponibili e i prestiti attivi.

- Ogni libro è identificato univocamente da un codice ISBN (stringa) e possiede un titolo e un numero totale di copie fisiche disponibili.
- Deve essere possibile registrare un prestito fornendo il nome dell'utente e l'ISBN. Se ci sono copie disponibili, il prestito viene accordato (restituendo **true**) e le copie diminuiscono. Se le copie sono esaurite, la richiesta viene inserita in una coda d'attesa e il metodo restituisce **false**.
- Deve essere possibile restituire un libro. Se c'è qualcuno in coda d'attesa per quell'ISBN, il libro viene automaticamente assegnato al primo utente in coda (senza aumentare le copie sullo scaffale).
- Implementare un metodo che, dato l'ISBN di un libro, restituisca il numero di persone attualmente in coda d'attesa.

Simulazione 3

Esercizio 1

Data la seguente porzione di programma rispondere, dando motivazione, alle domande corrispondenti:

```
int* f(int* arr, int size) {
    return arr + (size / 2);
}

int main() {
    int* matricola = new int[6] { /* Inserisci tua matricola */ };

    // 1. Le istruzioni seguenti sono corrette? Se si, cosa stampano?
    int* mid = f(matricola, 6);
    cout << mid[0] << endl;

    // 2. Le istruzioni seguenti sono corrette? Se si, cosa stampano?
    *(matricola + 2) = *(matricola + 4) + 1;
    cout << matricola[2] << endl;

    // 3. Le seguenti istruzioni sono corrette? Motivare la risposta.
    while (matricola != nullptr) {
        matricola++;
    }

    // 4. Scrivere sul foglio le istruzioni per deallocare la memoria.
    return 0;
}
```

Esercizio 2

Modellare una classe **TrackerAbitudini** per tenere traccia delle abitudini quotidiane di un utente. Di ogni abitudine (nome stringa immutabile) si vuole memorizzare l'obiettivo settimanale e lo *streak* attuale (numero di volte consecutive in cui l'obiettivo è stato raggiunto).

- La classe deve permettere di aggiungere nuove abitudini.
- Fornire un metodo `segna_completamento(string nome, bool raggiunto)`: se `raggiunto` è `true`, lo *streak* aumenta di 1; se è `false`, lo *streak* si azzerava tornando a 0.
- Definire un costruttore che prenda in input una mappa (`map<string, unsigned>`) per inizializzare massivamente diverse abitudini con i rispettivi obiettivi settimanali (partendo tutte con *streak* 0).
- Implementare una classe derivata **TrackerAbitudiniGamificato**, in cui ogni volta che uno *streak* raggiunge un multiplo di 5, l'utente guadagna 10 "Punti Esperienza" (XP). Esporre un metodo per leggere gli XP totali.

Simulazione 4

Esercizio 1

Data la seguente porzione di programma rispondere alle domande corrispondenti:

```
void mistero(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int* matricola = new int[6] { /* Inserisci tua matricola */ };

    // 1. La seguente istruzione e' corretta? Se si, che cosa stampa?
    mistero(&matricola[0], &matricola[5]);
    cout << matricola[0] << " " << matricola[5] << endl;

    // 2. La seguente istruzione e' corretta? Se si, che cosa stampa?
    int** ptr = &matricola;
    cout << *(*ptr + 1) << endl;

    // 3. Le seguenti istruzioni sono corrette? Motivare la risposta.
    int* t = new int;
    *t = matricola[2];
    delete t;
    cout << matricola[2] << endl;

    // 4. Selezionare TUTTE le operazioni necessarie per deallocare.
    return 0;
}
```

Esercizio 2

Un'app di e-commerce necessita di un sistema per gestire il "Carrello". Implementare una classe **Carrello** con le seguenti funzionalità:

- Aggiungere un prodotto specificandone nome e prezzo. Se il prodotto è già nel carrello, aggiornare il suo prezzo al valore più basso tra quello già memorizzato e quello nuovo.
- Un metodo `applica_sconto(float percentuale)` che riduce il prezzo di tutti gli articoli della percentuale indicata. Non è possibile che il prezzo scenda sotto lo 0.
- Un metodo `totale()` che restituisce la somma dei prezzi di tutti gli articoli presenti.
- Implementare una seconda classe **CarrelloPremium** che eredita da **Carrello**. Nel **CarrelloPremium**, ogni volta che `totale()` restituisce un valore superiore a 100 euro, viene automaticamente aggiunto un "Prodotto Omaggio" a prezzo 0 (se non è già presente).

Simulazione 5

Esercizio 1

Data la seguente porzione di programma rispondere alle domande corrispondenti:

```
int main() {
    int* matricola = new int[6] { /* Inserisci tua matricola */ };

    // 1. La seguente istruzione e' corretta? Se si, cosa stampa?
    int val = 0;
    for(int i = 0; i < 6; i++) {
        if(matricola[i] % 2 != 0) val += matricola[i];
    }
    cout << val << endl;

    // 2. Le seguenti istruzioni sono corrette? Se si, cosa stampano?
    int* p = matricola + 4;
    p[-2] = 99;
    cout << matricola[2] << endl;

    // 3. Perche' questo ciclo genera un errore a runtime o indefinito?
    for(int i = 0; i <= 6; i++) {
        matricola[i] = 0;
    }

    // 4. Scrivere le istruzioni per deallocare la memoria dinamica.
    return 0;
}
```

Esercizio 2

Modellare e implementare una classe `TorneoVideogiochi` che gestisca una classifica di giocatori. Ogni giocatore è identificato dal proprio *nickname* (stringa univoca).

- Costruttore che accetta in input una lista (`vector<string>`) di partecipanti iniziali, assegnando a ciascuno 0 punti.
- Un metodo `registra_partita(string vincitore, string perdente)` che aggiunge 3 punti al vincitore e sottrae 1 punto al perdente. Se uno dei due non è iscritto, l'operazione fallisce e restituisce `false` (altrimenti `true`). Il punteggio non scende mai sotto zero.
- Un metodo `classifica_top(unsigned k)` che restituisce un `vector<string>` con i nickname dei k giocatori con il punteggio più alto, in ordine decrescente.
- Definire un operatore `>` (maggiore) tra due istanze di `TorneoVideogiochi` che restituisce `true` se il primo torneo ha un numero totale di partecipanti strettamente maggiore del secondo.